

Curso Básico de Python:

Leitura e Manipulação de Dados



Ana Sophia; Lillya Maria

Universidade Estadual da Paraíba

25 de setembro de 2026

- 1 Introdução
- 2 Manipulação de Dados
- 3 Limpeza e Tratamento de Dados
- 4 Criação de Gráficos
- 5 Verificação Final e Exportação
- 6 Estudo de Caso

Introdução



Por que dados precisam ser tratados

- Dados reais raramente vêm prontos para análise;
- Erros de digitação, campos em branco, categorias inconsistentes são comuns;
- Uma base mal tratada leva a conclusões erradas.

Por que utilizar Python

- Automatiza tarefas repetitivas;
- É reprodutível;
- Documenta cada etapa da limpeza;
- Lida bem com bases grandes;
- Une limpeza, análise e gráficos no mesmo lugar.

- OpenDataSUS — portal com bases abertas do Ministério da Saúde;
- DATASUS / TabNet — principal repositório público de dados de saúde do Brasil;
- IBGE — dados demográficos e socioeconômicos complementares;
- OMS / OPAS (Global Health Observatory) — dados internacionais de saúde.

Manipulação de Dados



- É o processo de transformar, organizar, limpar e re-estruturar dados para análise;
- Envolve filtragem, seleção, agregação, ordenação, renomeação e junção de dados;
- **Objetivo:** preparar os dados para que possam ser usados tanto em análises estatísticas quanto em modelos de aprendizado de máquina, capazes de revelar padrões ocultos que a análise descritiva sozinha não identifica.

Pandas

é uma biblioteca open source voltada à manipulação e análise de dados, oferecendo estruturas como Series (uma coluna) e DataFrames (tabelas), além de recursos de leitura, limpeza e visualização de dados.

DataFrames

- estrutura de dados tabular bidimensional (linhas e colunas), como uma matriz;
- projetado para trabalhar com dados de forma prática e eficiente;
- permite adicionar, modificar, filtrar e agregar dados estruturados;
- uniformiza a importação de diversos formatos de arquivo (CSV, Excel, JSON, XML) em um único tipo de estrutura;

- **CSV**: texto simples com valores separados por vírgula ou ponto-e-vírgula;
- **Excel (XLS/XLSX)**: formato do Microsoft Excel, com múltiplas abas, fórmulas e formatação;
- **JSON**: estrutura hierárquica de chave-valor, muito usada em APIs web;
- **XML**: estrutura hierárquica baseada em tags, comum em sistemas legados e bases públicas.

```
# importar a biblioteca Pandas

import pandas as pd

# para ler os dados de um arquivo CSV
dados = pd.read_csv( 'pacientes.csv' )
# para ler os dados de um arquivo JSON
dados = pd.read_json( 'pacientes.json' )
# para ler os dados de um arquivo XML
dados = pd.read_xml( 'pacientes.XML' )
# para ler os dados de um arquivo Excel
dados = pd.read_excel( 'pacientes.xlsx' )
```

Separador (sep)

- define qual caractere separa as colunas no arquivo;
- bases brasileiras costumam usar ; em vez de , (porque a vírgula já é usada como separador decimal).

Pular linhas (skiprows)

- ignora as primeiras linhas do arquivo antes de começar a ler a tabela;
- útil quando o arquivo tem um título, metadado ou informação extra antes do cabeçalho real.

Codificação de caracteres (encoding)

- corrige problemas de acentuação (ex.: "saúde" aparecer como "saÃºde");
- codificações comuns: latin1, utf8, cp1252.

Selecionar colunas específicas (usecols)

- importa só as colunas indicadas (por posição ou nome), ignorando o resto;
- útil para bases grandes, quando só uma parte das colunas interessa.

```
dados = pd.read_csv('pacientes.csv', sep= ';' )  
dados = pd.read_csv('pacientes.csv', sep= ';' , skiprows= 1 )  
dados = pd.read_csv('pacientes.csv', sep= ';' , skiprows= 1 , encoding= 'latin1' )  
dados = pd.read_csv('pacientes.csv', usecols=[ 0 , 1 , 2 ])
```

```
# para excluir uma coluna já importada  
  
# axis = 0 para indicar uma linha  
# axis = 1 para indicar uma coluna  
dados = dados.drop('UBS', axis=1)
```

Limpeza e Tratamento de Dados

- é o documento que descreve cada campo de uma base: nome, tipo, definição, intervalos possíveis e categorias das variáveis.
- essencial para garantir qualidade e consistência na interpretação dos dados;

Variável	Tipo	Descrição
CO_ANOMES	Numérica	Ano e mês de referência
NO_MUNICIPIO	Categórica	Nome do município
SG_UF	Categórica	Sigla da UF
VL_INDICADOR_CALCULADO_MUN	Numérica	Indicador do município

Tabela: Exemplo de dicionário de dados

dados.columns

Mostra uma lista com os nomes das colunas.

```
Index(['co_anomes', 'co_ibge', 'vl_indicador_calculado_mun', 'co_uf',  
      'no_municipio', 'sg_uf', 'no_uf', 'co_regiao_brasil',  
      'no_regiao_brasil', 'sg_regiao_brasil', 'co_regiao_saude',  
      'no_regiao_saude', 'no_macro', 'co_macro', 'vl_indicador_calculado_rs',  
      'vl_indicador_calculado_ms', 'vl_indicador_calculado_uf',  
      'vl_indicador_calculado_reg', 'vl_indicador_calculado_br',  
      'vl_indicador_calculado_al', 'dt_competencia', 'dt_atualizacao',  
      'ds_unidade_medida', 'sg_granularidade', 'ds_granularidade'],  
      dtype='object')
```

dados.info()

Mostra as informações das variáveis importadas.

```
RangeIndex: 61273 entries, 0 to 61272  
Data columns (total 25 columns):  
#   Column                                Non-Null Count  Dtype  
---  ---  
0   co_anomes                             61273 non-null  int64  
1   co_ibge                               61273 non-null  int64  
2   vl_indicador_calculado_mun            61273 non-null  float64  
3   co_uf                                 61273 non-null  int64  
4   no_municipio                          61273 non-null  object  
5   sg_uf                                 61273 non-null  object  
6   no_uf                                 61273 non-null  object
```

```
# mostra as 5 primeiras linhas e 5 últimas linhas
dados
# mostra uma amostra aleatória de 10 linhas
dados.sample(10)
# mostra o início (topo) da lista de dados
dados.head()
# mostra o fim (cauda) da lista de dados
dados.tail()
```

Identificando duplicatas

- `keep=False` mostra todos os duplicados;
- `keep='first'/'last'` mostra apenas a primeira/última ocorrência;
- pode-se checar por duplicados em alguma coluna específica ou em conjuntos de colunas.

```
dados[dados.duplicated(['DT_COMPETENCIA'], keep=False)]
```

Removendo duplicatas

- pode-se apagar as linhas onde todas as colunas forem iguais ou selecionar linhas de um conjunto de colunas específicas.

```
dados.drop_duplicates()  
dados.drop_duplicates(subset=['DT_COMPETENCIA', 'DT_ATUALIZACAO'], keep='first', inplace=True)
```

`.isnull().sum()`

- retorna a quantidade de casos ausentes por variável.

`.dropna()`

- `how='any'`: remove a linha se qualquer variável estiver ausente;
- `how='all'`: remove só se todas as variáveis da linha estiverem ausentes;
- `subset=[...]`: aplica a regra apenas a colunas específicas

```
dados.isnull().sum()
dados_sem_nulos = dados.dropna(how = 'any')
dados_sem_linhas_vazias = dados.dropna(how = 'all')
dados_validos = dados.dropna(how = 'any', subset=['DT_COMPETENCIA'])
```

.fillna()

- substitui os valores ausentes de uma coluna por um valor definido;
- esse valor pode ser a média, mediana ou moda, cabe ao analista decidir a mais adequada para cada variável.

```
mediana = dados.glicemia.median()  
dados.glicemia = dados.glicemia.fillna(mediana)
```

Recodificando valores como ausentes

- consiste em identificar valores que representam ausência de informação e substituí-los por NaN (valor nulo);
- isso evita que esses valores sejam interpretados como dados válidos durante análises estatísticas e cálculos posteriores.

```
dados.creatinina = dados.creatinina.replace(0, np.nan)
```

- Outliers são valores que se destacam muito do padrão geral dos dados, podem ser muito acima ou muito abaixo do esperado para aquela variável;
- Pode ser um outlier de erro (falha de digitação, de medição ou de transmissão do dado) ou de caso extremo real;
- **.describe()** retorna um resumo estatístico de cada variável numérica: mínimo, máximo, média, mediana e quartis;
- Uma forma prática de identificar outliers é comparando os valores mínimo e máximo com a mediana e a média;
- Se o mínimo ou o máximo estiverem muito distantes da mediana, é um sinal de alerta.

Recodificação de valores

```
dados.sg_uf.replace('PB', 'Paraíba', inplace=True)
```

Arredondamento e conversão de datas

```
dados.glicemia = dados.glicemia.round(1)  
dados.data_admissao = pd.to_datetime(dados.data_admissao)  
dados.data_exame = pd.to_datetime(dados.data_exame, format='%d/%m/%Y')
```

Limpeza de dados

Remove espaços extras no início/fim de campos de texto, ajudando na padronização dos dados.

```
dados.no_municipio = dados.no_municipio.str.strip()
```

Maiores e menores valores

```
dados.nlargest(5, 'vl_indicador_calculado_mun')  
dados.nsmallest(5, 'vl_indicador_calculado_mun')
```

Filtro por condição

```
adultos = dados[dados.idade >= 18]  
adultos_recife = dados[(dados.idade >= 18) & (dados.cidade == 'Recife')]
```

Filtro por texto parcial

```
dados2 = dados[dados.no_municipio.str.contains('Campina', na=False)]
```

Coluna Condicional

cria uma nova coluna com valores definidos de acordo com uma ou mais condições.

```
dados.loc[dados.glicemia < 100, 'FAIXA_GLICEMIA'] = 'Normal'  
dados.loc[dados.glicemia >= 100, 'FAIXA_GLICEMIA'] = 'Alterada'
```

Coluna Calculada

cria uma nova coluna a partir de cálculos ou transformações de outras colunas.

```
dados = dados.assign(GLICEMIA_MG_L = dados.glicemia * 10)
```

Faixas com pd.cut()

- transforma uma variável contínua (ex.: idade) em faixas categóricas (0-17, 18-59, 60+).

Ajustando valores impossíveis para nulo

- valores que não fazem sentido para a variável são tratados como dados ausentes.

```
dados.loc[dados.idade < 0, 'idade'] = np.nan
```

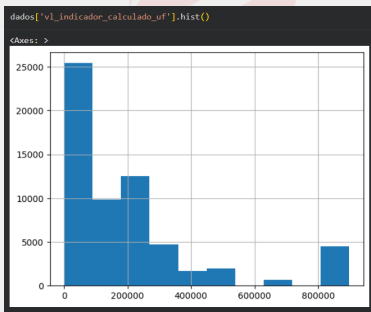
Criação de Gráficos



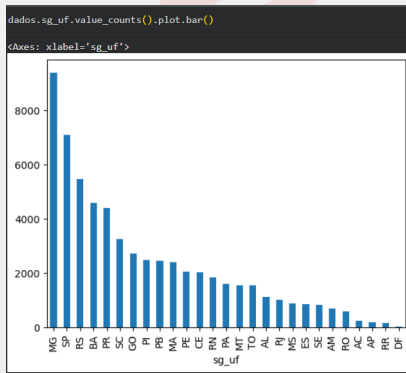
- Matplotlib é uma biblioteca de visualização de dados do Python, usada para transformar números em gráficos;
- Permite criar histogramas, gráficos de barras, linhas e dispersão;
- Serve dar uma segunda confirmação visual do que os números já mostraram, ajudando a enxergar padrões, concentrações e valores extremos que a tabela sozinha não deixa tão evidente.

```
import matplotlib.pyplot as plt
```

- Permite visualizar a distribuição dos valores de uma variável numérica por meio de um histograma;
- Mostra a concentração e dispersão dos dados;
- Ajuda a identificar padrões e valores extremos;
- Pode ser utilizado após o `.describe()` para complementar a análise estatística.



- O método `.value_counts()` conta a frequência de cada categoria, enquanto `.plot.bar()` apresenta essas frequências em um gráfico de barras;
- Útil para comparar a quantidade de registros entre categorias.



Verificação Final e Exportação

- Rode `.info()` e `.describe()` novamente;
- Confirme se cada variável possui o tipo adequado, se ainda existem dados não tratados ou se há possíveis outliers;
- A limpeza é uma etapa fundamental do projeto, pois garante a qualidade e a consistência dos dados, evitando erros que possam comprometer os resultados das análises.

- Após a limpeza e organização, os dados podem ser salvos em novos arquivos para utilização nas próximas etapas do projeto.

```
dados.to_csv('base_tratada.csv')  
dados.to_excel('base_tratada.xlsx')
```

- Permite salvar os dados em diferentes formatos, como CSV e Excel;
- A versão tratada deve receber um nome ou versão diferente, facilitando a organização e a recuperação dos dados originais.

Estudo de Caso



- aqui iremos aplicar nossos conhecimentos aprendidos nessa aula para tratar e gerar gráficos de uma base de dados fictícia.
- ela é propositalmente "suja": tem duplicatas, valores ausentes, outliers e texto mal formatado;
- todos os assuntos aprendidos poderão ser utilizados nessa base de dados.

Variável	Tipo	Descrição
NOME	Texto	Nome do paciente
SEXO	Categórica	Sexo (F/M; 9 = ignorado)
IDADE	Numérica	Idade em anos
PESO_KG	Numérica	Peso em quilogramas
UBS	Categórica	Unidade Básica de Saúde do atendimento
DT_ATENDIMENTO	Data	Data do atendimento

Tabela: Dicionário de dados de pacientes.csv

- Para realizarmos a importação da base de dados em CSV:
■ `dados = pd.read_csv('pacientes.csv', sep=';')`

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

dados = pd.read_csv('pacientes.csv', sep=';')
```

Para tratar essa duplicata, podemos decidir usar o 'first' dados = dados.drop_duplicates(keep='first')

```
dados[dados.duplicated(keep=False)]
```

	NOME	SEXO	IDADE	PESO_KG	UBS	DT_ATENDIMENTO
0	Maria da Silva	F	34.0	68.5	UBS Central	2026-03-02
1	Joao Pedro Souza	M	45.0	82.0	UBS Central	2026-03-02
6	Maria da Silva	F	34.0	68.5	UBS Central	2026-03-02
17	Joao Pedro Souza	M	45.0	82.0	UBS Central	2026-03-02

```
dados = dados.drop_duplicates(keep='first')
dados.shape
```

```
(38, 6)
```

- Aqui percebemos que temos dois dados ausentes:
- SEXO usa o código 9 como "ignorado" → recodificado para NaN;
- PESO_KG ausente → preenchido com a mediana;
- IDADE ausente → linha removida.

```
# SEXO usa 9 como codigo de "ignorado" -> recodificar para NaN
dados['SEXO'] = dados['SEXO'].replace(9, np.nan)
```

```
# PESO_KG ausente -> preenchido com a mediana (evita distorcao por outliers)
mediana_peso = dados['PESO_KG'].median()
dados['PESO_KG'] = dados['PESO_KG'].fillna(mediana_peso)
```

```
# IDADE ausente -> linhas removidas (nao ha estimativa confiavel para idade)
dados = dados.dropna(subset=['IDADE'])
dados.isnull().sum()
```

- Antes com dados ausentes presentes:

```
dados.isnull().sum()
```

0

NOME

0

SEXO

0

IDADE

2

PESO_KG

2

UBS

0

DT_ATENDIMENTO 0

dtype: int64

- Após o tratamento retiramos os dados ausentes:

```
dados.isnull().sum()
```

...	0
NOME	0
SEXO	0
IDADE	0
PESO_KG	0
UBS	0
DT_ATENDIMENTO	0

dtype: int64

- vemos com o `.describe()` revelando IDADE de 200 anos e -5 anos, além disso, dados com PESO_KG igual a 0;
- percebe-se os outliers e decide-se tratar dessa maneira:
- vamos tratar fazendo com que valores impossíveis virem NaN e depois são preenchidos com a mediana.

```
# IDADE negativa ou muito acima do plausível -> tratada como ausente
dados.loc[(dados.IDADE < 0) | (dados.IDADE > 110), 'IDADE'] = np.nan
# PESO_KG igual a 0 -> tratada como ausente (erro de registro)
dados.loc[dados.PESO_KG == 0, 'PESO_KG'] = np.nan

dados['IDADE'] = dados['IDADE'].fillna(dados['IDADE'].median())
dados['PESO_KG'] = dados['PESO_KG'].fillna(dados['PESO_KG'].median())
dados.describe()
```

■ Antes:

```
dados.describe()
```

	IDADE	PESO_KG
count	36.000000	36.000000
mean	44.750000	65.375000
std	32.451612	18.015428
min	-5.000000	0.000000
25%	28.500000	60.025000
50%	41.500000	68.850000
75%	55.750000	76.600000
max	200.000000	95.600000

■ Depois:

	IDADE	PESO_KG
count	36.000000	36.000000
mean	41.638889	67.297222
std	16.863998	14.108933
min	8.000000	30.400000
25%	29.750000	61.300000
50%	41.500000	69.200000
75%	53.500000	76.600000
max	71.000000	95.600000

- para começar nossa formatação desejada, podemos realizar algumas mudanças:
- recodifica SEXO (F/M → Feminino/Masculino)
- arredonda PESO_KG
- converte DT_ATENDIMENTO para data
- remove espaços de NOME e UBS com `.str.strip()`;

```
dados['SEXO'] = dados['SEXO'].replace('F', 'Feminino')
dados['SEXO'] = dados['SEXO'].replace('M', 'Masculino')

dados['PESO_KG'] = dados['PESO_KG'].round(1)
dados['DT_ATENDIMENTO'] = pd.to_datetime(dados['DT_ATENDIMENTO'])

dados['NOME'] = dados['NOME'].str.strip()
dados['UBS'] = dados['UBS'].str.strip()
dados.head()
```

- para treinar a criação de novas variáveis com python em uma base de dados, podemos criar uma de faixa de peso, peso em gramas e uma de faixa etária com `pd.cut()`.

```
# Coluna condicional
dados.loc[dados.PESO_KG < 60, 'FAIXA_PESO'] = 'Baixo'
dados.loc[dados.PESO_KG >= 60, 'FAIXA_PESO'] = 'Normal'

# Coluna calculada
dados = dados.assign(PESO_G = dados.PESO_KG * 1000)

# Faixas com pd.cut()
dados['FAIXA_ETARIA'] = pd.cut(
    dados.IDADE,
    bins=[0, 17, 59, 120],
    labels=['0-17', '18-59', '60+']
)
dados.head()
```

	NOME	SEXO	IDADE	PESO_KG	UBS	DT_ATENDIMENTO	FAIXA_PESO	PESO_G	FAIXA_ETARIA
0	Maria da Silva	Feminino	34.0	68.5	UBS Central	2026-03-02	Normal	68500.0	18-59
1	Joao Pedro Souza	Masculino	45.0	82.0	UBS Central	2026-03-02	Normal	82000.0	18-59
2	Ana Paula Lima	Feminino	41.5	75.3	UBS Norte	2026-03-03	Normal	75300.0	18-59
3	Carlos Eduardo	Masculino	29.0	69.2	UBS Sul	2026-03-03	Normal	69200.0	18-59
4	Francisca Alves	Feminino	61.0	59.8	UBS Norte	2026-03-04	Baixo	59800.0	60+

- nossas novas variáveis ficam dessa maneira:

PESO_G	FAIXA_ETARIA
68500.0	18-59
82000.0	18-59
75300.0	18-59
69200.0	18-59
59800.0	60+

- aqui podemos também criar filtros, como por exemplo:
- um filtro por condição: idosos ($IDADE \geq 60$) atendidos na UBS Norte;

```
# Filtro por condição: idosos atendidos na UBS Norte
idosos_ubs_norte = dados[(dados.IDADE >= 60) & (dados.UBS == 'UBS Norte')]
idosos_ubs_norte
```

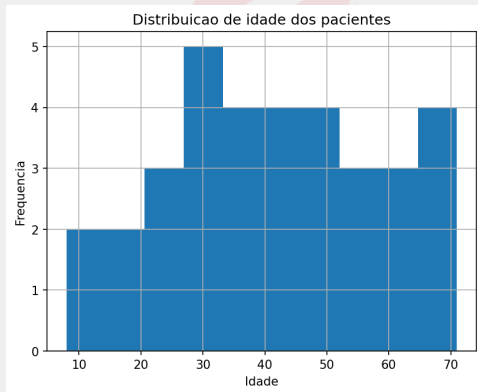
	NOME	SEXO	IDADE	PESO_KG	UBS	DT_ATENDIMENTO
4	Francisca Alves	Feminino	61.0	59.8	UBS Norte	2026-03-04
11	Marcos Vinicius	Masculino	71.0	69.2	UBS Norte	2026-03-07
21	Marcos Vinicius	Masculino	71.0	84.1	UBS Norte	2026-03-07
33	Vinicius Teixeira	Masculino	68.0	87.2	UBS Norte	2026-03-17
39	Wagner Duarte	9	63.0	72.1	UBS Norte	2026-03-20

- agora com a adição dessas novas variáveis, mudanças na formatação e filtros:

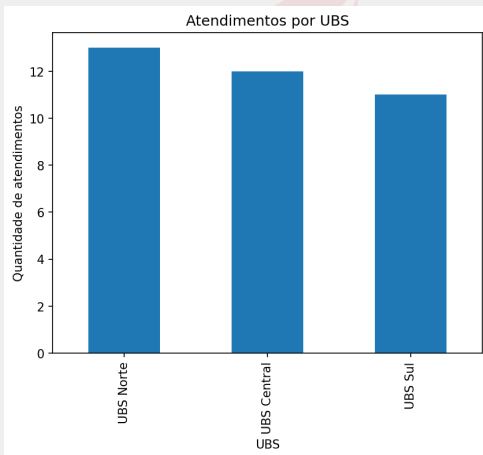
	NOME	SEXO	IDADE	PESO_KG	UBS	DT_ATENDIMENTO	FAIXA_PESO	PESO_G	FAIXA_ETARIA
0	Maria da Silva	Feminino	34.0	68.5	UBS Central	2026-03-02	Normal	68500.0	18-59
1	Joao Pedro Souza	Masculino	45.0	82.0	UBS Central	2026-03-02	Normal	82000.0	18-59
2	Ana Paula Lima	Feminino	41.5	75.3	UBS Norte	2026-03-03	Normal	75300.0	18-59
3	Carlos Eduardo	Masculino	29.0	69.2	UBS Sul	2026-03-03	Normal	69200.0	18-59
4	Francisca Alves	Feminino	61.0	59.8	UBS Norte	2026-03-04	Baixo	59800.0	60+

- alguns exemplos de gráficos:

```
dados['IDADE'].hist()  
plt.title('Distribuicao de idade dos pacientes')  
plt.xlabel('Idade')  
plt.ylabel('Frequencia')  
plt.show()
```



```
dados['UBS'].value_counts().plot.bar()  
plt.title('Atendimentos por UBS')  
plt.xlabel('UBS')  
plt.ylabel('Quantidade de atendimentos')  
plt.show()
```



- após `.info()` e `.describe()` confirmarem a limpeza, a base é exportada:

```
dados.to_csv('pacientes_tratada.csv')
```

```
dados.to_excel('pacientes_tratada.xlsx')
```

Obrigado!

- [1] J. d. S. Gaspar, Z. S. N. Reis, I. J. R. d. Oliveira, A. P. C. d. Silva e C. d. S. Dias, *Introdução à Análise de Dados em Saúde com Python*. Belo Horizonte: Universidade Federal de Minas Gerais, 2023, ISBN: 978-65-86593-18-1. DOI: 10.5281/zenodo.7865448